Department of Software Engineering
Faculty of Science & Information Technology

UniExpert

AI-Powered Expert Discovery and Academic Collaboration Platform

Software Requirements Specification (SRS)

Standard Conforming to IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018

Course: SE 331 — Software Engineering Design Capstone Project

Semester: Fall 2026 · **Duration:** October – December 2026

Prepared By (Student Team)

Abu Zafor
Student ID: 232-35-331

Anamul Haque
Student ID: 232-35-285

Supervised By

Mr. Md. Shohel Arman
Assistant Professor
Department of Software Engineering

Date of Submission: October 12, 2026

Document Control and Revision History

Version	Date	Author(s)	Summary of Major Changes	Status
0.1	Oct 02, 2026	A. Zafor, A. Haque	Initial requirements elicitation and scope drafting.	Draft
0.5	Oct 07, 2026	A. Zafor, A. Haque	Complete IEEE 830 functional and non-functional requirements.	Reviewed
1.0	Oct 12, 2026	A. Zafor, A. Haque	Baseline SRS document submitted for Capstone committee review.	Approved

Role	Name & Title	Affiliation	Signature / Date
Lead Author	Abu Zafor (Student ID: 232-35-331)	Dept. of Software Engineering	Verified / Oct 12, 2026
Co-Author	Anamul Haque (Student ID: 232-35-285)	Dept. of Software Engineering	Verified / Oct 12, 2026
Faculty Supervisor	Mr. Md. Shohel Arman (Assistant Professor)	Dept. of Software Engineering	Verified / Oct 12, 2026

Executive Specification Overview

This Software Requirements Specification (SRS) establishes the formal baseline for the engineering, verification, and evaluation of **UniExpert**. UniExpert is an intelligent web-based discovery platform engineered to resolve the pervasive academic silo challenge in higher education. It parses natural language student research proposals, extracts skill entities via zero-shot transformer classifiers, indexes multi-modal expert profiles via dense embeddings in vector databases, executes multi-criteria hybrid ranking, and coordinates stateful collaboration lifecycles.

Contents

Document Control and Revision History **1**

1 Introduction **4**
1.1 Purpose of the Document . . . 4
1.2 Document Conventions . . . 4
1.3 Intended Audience and Reading Suggestions . . . 4
1.4 Project Scope . . . 4
1.5 Definitions, Acronyms, and Abbreviations . . . 5
1.6 References . . . 5

2 Overall Description **6**
2.1 Product Perspective . . . 6
2.2 Product Functions Overview . . . 6
2.3 User Classes and Characteristics . . . 6
2.4 Operating Environment . . . 7
2.5 Design and Implementation Constraints . . . 7
2.6 Assumptions and Dependencies . . . 7

3 External Interface Requirements **9**
3.1 User Interfaces (UI) . . . 9
3.2 Hardware Interfaces . . . 9
3.3 Software Interfaces . . . 9
3.4 Communications Interfaces . . . 10

4 System Features and Functional Requirements **11**
4.1 Module 1: Authentication and Identity Management . . . 11
4.2 Module 2: Profile Engineering and Taxonomy Management . . . 11
4.3 Module 3: Natural Language Problem Ingestion and NLP Extraction . . . 12
4.4 Module 4: Semantic Matching and Hybrid Recommendation Engine . . . 13
4.5 Module 5: Multidisciplinary Team Formation and Gap Analysis . . . 15
4.6 Module 6: Collaboration Lifecycle and Workspace Management . . . 15
4.7 Module 7: Feedback, Governance, and Administration . . . 17

5 Non-Functional Requirements (NFRs) **18**
5.1 Performance and Latency Requirements . . . 18
5.2 Security and Confidentiality Requirements . . . 18
5.3 Reliability and Availability Requirements . . . 18
5.4 Usability and Accessibility Requirements . . . 19

6 Data Requirements and Conceptual Schema **20**
6.1 Conceptual Data Model . . . 20
6.2 Data Dictionary . . . 20

7 Requirements Traceability Matrix (RTM) **22**

8 Verification and Validation Plan **23**
8.1 Unit and Integration Testing Protocol . . . 23
8.2 Information Retrieval (IR) Benchmarking Protocol . . . 23
8.3 User Acceptance Testing (UAT) . . . 23

9 Committee Review and Sign-Off **23**

List of Figures

1 UniExpert System Context Diagram 6
2 UniExpert High-Level Functional Decomposition 6
3 UniExpert Conceptual Entity-Relationship Diagram 20

List of Tables

1 Glossary of Technical Terms and Acronyms 5
2 User Classes and Operational Profiles 7
3 Software Interfaces and Protocols 10
4 Performance Benchmarks and Acceptance Criteria 18
5 Relational Database Data Dictionary 21
6 Requirements Traceability Matrix (RTM) 22

1 Introduction

1.1 Purpose of the Document

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, rigorous, and unambiguous description of the software requirements for **UniExpert** (AI-Powered Expert Discovery and Academic Collaboration Platform). This document specifies both functional and non-functional requirements, external interfaces, system constraints, domain models, and validation criteria. It serves as the formal contractual baseline between the student engineering team, the academic supervisor Mr. Md. Shohel Arman, and the capstone review committee for course **SE 331 (Software Engineering Design Capstone Project)** during the Fall 2026 semester.

1.2 Document Conventions

This SRS conforms to the IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018 guidelines. The following conventions are used throughout:

- **Requirement Identifiers:** Functional requirements are designated with the prefix `FR-[Module]-[Number]` (e.g., `FR-AUTH-01`).
- **Non-Functional Identifiers:** Non-functional requirements are designated with the prefix `NFR-[Category]-[Number]` (e.g., `NFR-PERF-01`).
- **Priority Tags:** Prioritization follows the MoSCoW framework:
  - **[M]** *Must-Have:* Mandatory core functionality for the Minimum Viable Product (MVP).
  - **[S]** *Should-Have:* High-priority capabilities required for comprehensive evaluation.
  - **[C]** *Could-Have:* Desirable enhancements targeted if time and compute permit.
  - **[W]** *Won't-Have (this release):* Explicitly deferred out-of-scope capabilities.
- **Normative Keywords:** The key words **SHALL**, **SHOULD**, and **MAY** are interpreted in accordance with RFC 2119.

1.3 Intended Audience and Reading Suggestions

This document is structured for the following stakeholders:

- **Capstone Evaluation Committee:** Focus on Section 2 (Product Functions), Section 4 (System Features), and Section 7 (Traceability).
- **Software Architects and Developers:** Focus on Section 3 (External Interfaces), Section 4 (Requirements Cards), and Section 6 (Data Models).
- **Quality Assurance and Testers:** Focus on Section 5 (Non-Functional Benchmarks) and Section 8 (Verification and Validation Plan).

1.4 Project Scope

The UniExpert platform addresses the systemic problem in university environments where students undertaking capstone projects, theses, and interdisciplinary research struggle to discover and initiate collaboration with qualified academic and industry mentors. UniExpert transforms expert discovery from a rigid keyword search into a problem-first, natural language workflow.

The core objectives of the system are:

(1) Ingest unstructured, natural language project statements submitted by students.
(2) Automatically extract domain classifications, technical skill prerequisites, and research scopes using zero-shot NLP models.

(3) Maintain a verified, multi-tier corpus of faculty, graduate researchers, and industry specialists.
(4) Compute multi-criteria compatibility scores combining semantic cosine similarity, skill taxonomy overlap, research interest alignment, credential authority, and schedule availability.
(5) Produce human-interpretable Explainable AI (XAI) justifications for every recommendation.
(6) Form multidisciplinary advisory teams using set-cover optimization to detect and resolve team skill gaps.
(7) Enforce a stateful collaboration lifecycle through an auditable Finite State Machine (FSM).

1.5 Definitions, Acronyms, and Abbreviations

Table 1 defines the domain-specific terms and acronyms utilized in this specification.

Table 1: Glossary of Technical Terms and Acronyms

Term / Acronym	Full Expansion	Operational Definition in UniExpert
SRS	Software Requirements Specification	This formal specification document.
EFS	Expert Finding System	An information retrieval system that identifies people with expertise in a topic.
HNSW	Hierarchical Navigable Small World	Graph-based indexing algorithm used in Qdrant for approximate nearest neighbor vector search.
XAI	Explainable Artificial Intelligence	Subsystem that articulates human-readable rationales for expert recommendations.
FSM	Finite State Machine	Formal state transition model governing collaboration proposals.
RBAC	Role-Based Access Control	Authorization framework governing students, experts, and administrators.
BM25	Best Matching 25	Classical probabilistic keyword retrieval ranking function used as baseline.
NDCG	Normalized Discounted Cumulative Gain	Standard ranking metric assessing recommendation quality graded by relevance.
ORCID	Open Researcher and Contributor ID	Persistent digital identifier disambiguating academic authors.
JWT	JSON Web Token	Compact, URL-safe means of representing claims between two parties.

1.6 References

[1] IEEE Computer Society, *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.
[2] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, 2018.
[3] K. Balog, *Expertise Retrieval: A Survey*, Foundations and Trends in Information Retrieval, vol. 6, no. 2–3, pp. 127–256, 2012.
[4] N. Reimers and I. Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, in Proc. EMNLP 2019.

2 Overall Description

2.1 Product Perspective

UniExpert is a self-contained, cloud-deployable web application with dedicated external interfaces to university identity providers, public academic bibliometric repositories (ORCID, Google Scholar), and enterprise SMTP services. The system operates in a multi-tier microservice-inspired architecture, decoupling the compute-intensive AI inference pipeline from user-facing CRUD workloads.

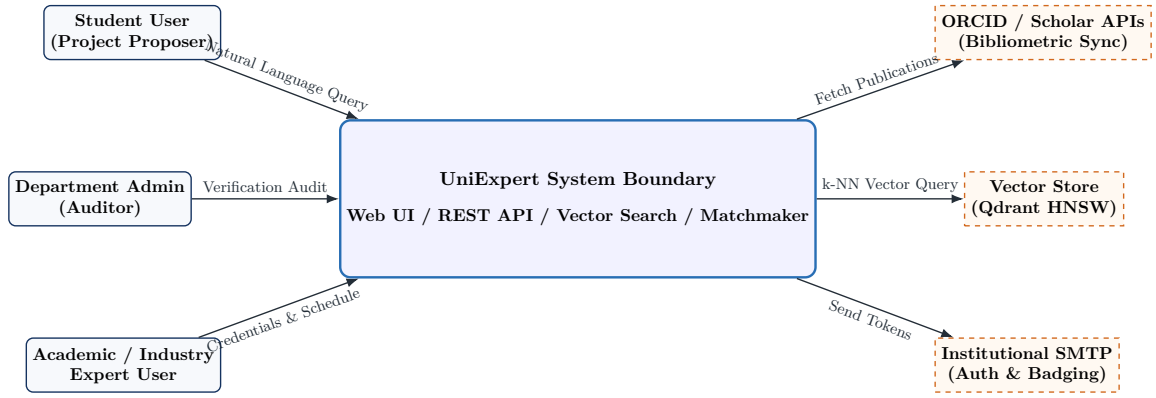


Figure 1: UniExpert System Context Diagram

2.2 Product Functions Overview

Figure 2 synthesizes the top-level functional hierarchy of UniExpert across its four core subsystems.

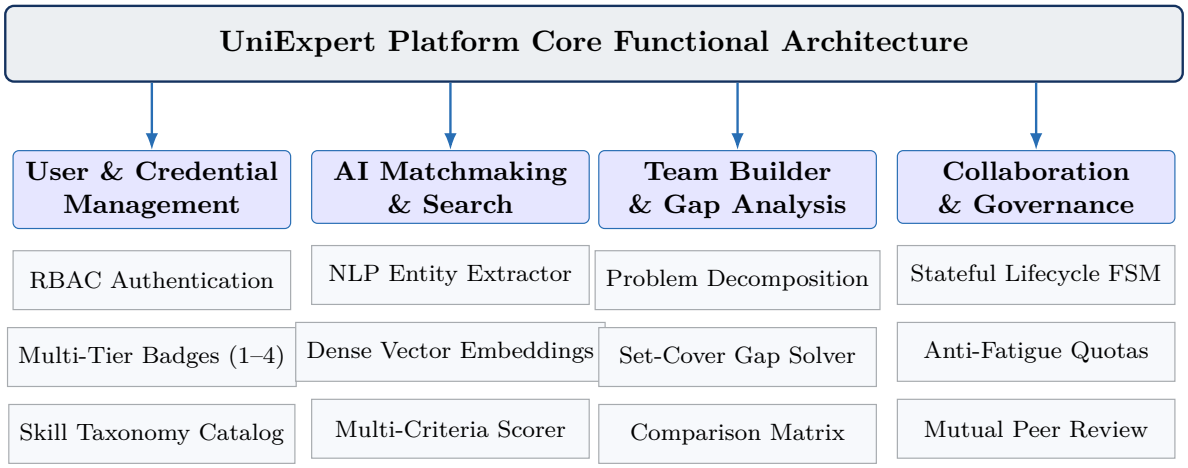


Figure 2: UniExpert Functional Subsystem Architecture

2.3 User Classes and Characteristics

UniExpert serves four distinct categories of users with varying privileges and operational objectives, summarized in Table 2.

Table 2: User Classes and Operational Profiles

User Class	Technical Profile	Key Responsibilities & Permissions	Frequency
Student Researcher	Moderate web familiarity; novice in taxonomy	Submits project proposals; reviews AI recommendations; creates multidisciplinary teams; sends collaboration requests.	Daily during semester
Academic Expert (Professor / Researcher)	High academic domain expertise	Publishes research skills; toggles availability blackouts; reviews project requests; collaborates in workspaces.	Weekly
Industry Mentor	High professional software / engineering skill	Offers industry perspective; sets consultation capacity; evaluates applied capstone feasibility.	Bi-weekly
Department Admin	High technical and administrative authorization	Verifies institutional documentation; resolves user disputes; inspects system telemetry and IR performance.	Periodic

2.4 Operating Environment

The software shall operate reliably within the following environments:

- **Client-Side:** Modern evergreen web browsers (Chromium $\geq$ 118, Mozilla Firefox $\geq$ 120, Apple Safari $\geq$ 17, Microsoft Edge $\geq$ 118) across desktop, tablet, and mobile displays.
- **Backend Server:** Linux OS (Ubuntu 22.04 LTS or Debian 12) running Python 3.11+ ASGI asynchronous web services (FastAPI / Uvicorn).
- **Data Stores:** PostgreSQL 16 relational database with ACID guarantees; Qdrant 1.8+ dedicated vector database with HNSW cosine distance indexing; Redis 7.2 in-memory cache and rate-limit broker.
- **Virtualization & Deployment:** Docker Engine 24.0+ and Docker Compose v2 for containerized local development and cloud production deployment.

2.5 Design and Implementation Constraints

(1) Compute Constraints: The AI inference engine must execute zero-shot skill extraction and embedding generation using CPU-optimized architectures (ONNX Runtime / FastEmbed) without requiring dedicated multi-GPU cloud instances.

(2) Security & Compliance: Student email addresses, draft project concepts, and direct messages must be protected using TLS 1.3 in transit and AES-256 at rest, respecting FERPA guidelines.

(3) Academic Integrity: The system shall not write or synthesize academic project code or thesis literature; its sole AI function is discovery, entity extraction, ranking, and matchmaking.

(4) Zero License Costs: The platform must strictly utilize permissive open-source libraries (MIT, Apache 2.0, BSD) ensuring no proprietary licensing liabilities.

2.6 Assumptions and Dependencies

- The university possesses an active SMTP relay or provides access to institutional email domains (`.edu`) for automated credential validation.

- Public research metadata can be harvested via open ORCID and Google Scholar endpoints within allowable rate limits.
- Participating experts voluntarily register and configure their availability status.

3 External Interface Requirements

3.1 User Interfaces (UI)

The UniExpert graphical interface shall be designed according to modern usability principles (WCAG 2.1 AA compliance):

- **Design System:** Cohesive design system built with clean typography, responsive CSS Grid / Flexbox layouts, and high-contrast color tokens.
- **Responsive Breakpoints:** Desktop ($\geq$ 1200px), Tablet (768px $-$ 1199px), and Mobile ($<$ 768px).
- **Interactive Views:**
  - **(a)** *Project Ingestion Console:* Freeform rich-text input with live entity highlighting showing extracted skills and suggested taxonomy tags.
  - **(b)** *Recommendation Explorer:* Side-by-side candidate cards displaying overall compatibility score, sub-scores, verified badges, and XAI justification tooltips.
  - **(c)** *Team Assembly Workbench:* Visual drag-and-drop panel computing collective skill coverage in real-time.
  - **(d)** *Collaboration FSM Dashboard:* Kanban-style visual pipeline representing sent, pending, accepted, and completed requests.

3.2 Hardware Interfaces

No dedicated proprietary hardware interfaces are required. The system communicates over standard TCP/IP networking hardware. Minimal recommended host hardware comprises:

- CPU: 4 vCPUs (x86-64 or ARM64 architecture).
- RAM: 8 GB memory minimum (16 GB recommended for high concurrent embedding loads).
- Storage: 40 GB NVMe SSD persistent storage for database volumes.

3.3 Software Interfaces

Table 3 enumerates all external and intra-system software interfaces.

Table 3: Software Interfaces and Protocols

Component	Interface Type	Protocol / Format	Purpose and Payload
PostgreSQL 16	Relational DB	TCP / asyncpg / SQL	Stores user accounts, profiles, project records, reviews, and audit logs.
Qdrant Engine	Vector Database	gRPC / REST / JSON	Stores 384-dimensional dense embeddings; executes cosine HNSW k-NN searches.
Redis 7	In-Memory Store	TCP / RESP protocol	Manages active user sessions, token blocklists, and API rate-limiting buckets.
FastEmbed / ONNX	Embedded ML	Python API	Generates `all-MiniLM-L6-v2` embeddings locally with sub-millisecond CPU latency.
ORCID API	Public Web API	HTTPS / REST / JSON	Fetches public author publication histories, DOI lists, and institutional affiliations.
Mail Server	Transactional Mail	SMTP / TLS (Port 587)	Dispatches account activation tokens, verification links, and invitation notifications.

3.4 Communications Interfaces

- **Client-Server Communication:** All web traffic shall be encrypted using TLS 1.3 over HTTPS. Unencrypted HTTP requests shall be rejected with HTTP 301 redirects.
- **API Communication:** RESTful endpoints follow JSON API conventions; API error responses adhere to RFC 7807 (Problem Details for HTTP APIs).
- **Real-Time Workspace Messaging:** Secured WebSockets (`wss://`) provide bi-directional event notifications for collaboration state transitions and instant chat updates.

4 System Features and Functional Requirements

This section details the 20 functional requirements grouped into 7 modular subsystems.

4.1 Module 1: Authentication and Identity Management

FR-AUTH-01: User Registration and Institutional Verification

Priority:	[M] Must-Have
Description:	The system SHALL allow students, academic experts, and industry mentors to register unique accounts. Academic experts registering with an institutional email (`.edu` or departmental domain) SHALL automatically trigger an automated cryptographic activation token.
Input:	Full Name, Email, Password (min 10 chars, mixed case, numeric, special), User Role.
Output:	Account creation confirmation, verification email with expiring HMAC token (24-hour TTL).
Precondition:	Email address is not previously registered in the system.
Postcondition:	Unverified user record created in PostgreSQL with status `PENDING_VERIFICATION`.

FR-AUTH-02: Role-Based Access Control (RBAC) and Session Management

Priority:	[M] Must-Have
Description:	The system SHALL enforce strict RBAC across four distinct scopes: `STUDENT`, `EXPERT`, `ADMIN`, and `AUDITOR`. Sessions SHALL be maintained using short-lived JWT access tokens (15-minute TTL) and secure HTTP-only refresh tokens (7-day TTL).
Input:	User login credentials (Email, Password).
Output:	Access JWT with encoded scopes; refresh token stored in encrypted cookie.
Precondition:	User email has been verified.
Postcondition:	Authentication token issued; session logged with IP and User-Agent telemetry.

4.2 Module 2: Profile Engineering and Taxonomy Management

FR-PROF-03: Student Academic Profile Management

Priority:	[M] Must-Have
Description:	The system SHALL provide students with a comprehensive profile dashboard capturing university name, student ID, department, current degree program, research interests, and previous project repositories.
Input:	Academic metadata, biography, CV attachment (PDF max 5MB), GitHub / LinkedIn URLs.
Output:	Rendered student profile card; searchable profile entity in database.
Precondition:	Authenticated with `STUDENT` role.
Postcondition:	Profile saved and associated with project listings.

FR-PROF-04: Multi-Tier Expert Credential Verification Workflow

Priority:	[M] Must-Have
Description:	The system SHALL support a four-tier progressive verification badge framework: 1. **Tier 1 (Email):** Verified institutional `.edu` email. 2. **Tier 2 (Bibliometric):** Verified ORCID ID with ≥ 2 indexed publications. 3. **Tier 3 (Institutional):** Departmental appointment letter reviewed by Admin. 4. **Tier 4 (Industry):** Corporate domain email with verified professional credentials.
Input:	Verification artifacts (ID card scan, appointment letter, ORCID token).
Output:	Public profile badges: `[Verified Faculty]`, `[Verified Researcher]`, `[Industry Mentor]`.
Precondition:	Authenticated with `EXPERT` role.
Postcondition:	Credential authority weight $\mathcal{V}(e)$ updated in ranking algorithm.

FR-PROF-05: Hierarchical Academic Skill and Domain Taxonomy

Priority:	[M] Must-Have
Description:	The system SHALL maintain a structured, multi-level academic taxonomy spanning Computer Science, Software Engineering, Electrical Engineering, Biotechnology, Agriculture, Business, and Data Science. It SHALL support autocomplete and alias mapping (e.g., mapping "CV" to "Computer Vision").
Input:	Search keywords or taxonomy navigation queries.
Output:	Standardized taxonomy entity nodes with parent, child, and related skill links.
Precondition:	System taxonomy seeded in database.
Postcondition:	Query skill mapped to canonical taxonomy ID.

4.3 Module 3: Natural Language Problem Ingestion and NLP Extraction

FR-NLP-06: Unstructured Project Statement Ingestion

Priority:	[M] Must-Have
Description:	The system SHALL accept free-text project problem descriptions from students (minimum 50 words, maximum 2,000 words). The submission form SHALL accept project title, target domain, expected methodology, and project timeline.
Input:	Free-text research statement, target domain category, project type (Capstone, Thesis, Research).
Output:	Structured project record assigned a unique UUID.
Precondition:	Authenticated student user.
Postcondition:	Asynchronous NLP parsing event dispatched to background task worker.

FR-NLP-07: Automated Skill and Domain Entity Extraction

Priority:	[**M**] Must-Have
Description:	The system SHALL parse project descriptions using zero-shot classification and dictionary-guided named entity recognition (NER). It SHALL extract: (1) Primary Domain; (2) Secondary Domains; (3) Core Technical Prerequisites; and (4) Methodological Paradigms.
Input:	Raw unstructured project text string.
Output:	JSON array of identified skills with confidence scores $\in [0, 1]$.
Precondition:	NLP model pipeline initialized and warm in memory.
Postcondition:	Extracted entities linked to project record in relational store.

FR-NLP-08: Interactive Student Requirement Refinement

Priority:	[**S**] Should-Have
Description:	The system SHALL present extracted skills and domain tags to the student in an interactive confirmation interface. The student SHALL have the ability to remove spurious tags, adjust importance weights (Must-Have vs. Nice-to-Have), and manually append additional skills.
Input:	Student adjustments (tag deletions, additions, weight modifications).
Output:	Finalized requirement vector $\mathcal{K}_u$ with associated weights w_k.
Precondition:	Automated NLP extraction completed.
Postcondition:	Refined requirement vector committed as input to matchmaker.

4.4 Module 4: Semantic Matching and Hybrid Recommendation Engine

FR-REC-09: Dense Semantic Embedding Generation and Vector Indexing

Priority:	[**M**] Must-Have
Description:	The system SHALL transform finalized project problem statements and expert research biographies into 384-dimensional dense semantic vectors using the `all-MiniLM-L6-v2` embedding model. Vectors SHALL be indexed in Qdrant with HNSW graphs ($M = 16, \text{ef_construct} = 100$).
Input:	Canonical text representations of projects and expert profiles.
Output:	384-dimensional unit-normalized embedding vector $\vec{v} \in \mathbb{R}^{384}$.
Precondition:	Text preprocessing and stop-word normalization executed.
Postcondition:	Vector stored in Qdrant collection with payload containing profile ID.

FR-REC-10: Multi-Criteria Hybrid Scoring Engine

Priority:	[M] Must-Have
Description:	The system SHALL compute an overall composite compatibility score $\mathcal{S}(u, e) \in [0, 1]$ between student project u and expert candidate e using the formal multi-criteria formulation: $$\mathcal{S}(u,e) = w_1 \cdot \mathcal{D}(u,e) + w_2 \cdot \mathcal{K}(u,e) + w_3 \cdot \mathcal{R}(u,e) + w_4 \cdot \mathcal{V}(e) + w_5 \cdot \mathcal{A}(u,e) \quad (1)$$ subject to $\sum_{i=1}^{5} w_i = 1.0$ (baseline configuration: $w_1 = 0.30, w_2 = 0.25, w_3 = 0.20, w_4 = 0.15, w_5 = 0.10$).
Input:	Student project vector, extracted skill list, candidate expert profiles.
Output:	Ranked list of candidate experts ordered by descending $\mathcal{S}(u, e)$.
Precondition:	Candidate expert matches availability and domain criteria.
Postcondition:	Top-K recommendations ($K = 10$) rendered to student.

FR-REC-11: Explainable AI (XAI) Justification Generation

Priority:	[M] Must-Have
Description:	For every recommended expert, the system SHALL synthesize a natural-language explanatory rationale detailing: • Direct overlapping skill sets (e.g., "Matches 4 out of 5 required skills: PyTorch, Computer Vision"). • Relevant recent publications or project history aligned with the student's problem. • Current availability status and preferred collaboration modes.
Input:	Sub-score breakdown from Equation 1 and overlapping entity sets.
Output:	Human-readable justification card displayed beneath candidate card.
Precondition:	Hybrid score calculated.
Postcondition:	Explanatory metadata cached for frontend rendering.

FR-REC-12: Dynamic Faceted Filtering and Re-ranking

Priority:	[M] Must-Have
Description:	The system SHALL allow students to dynamically filter and re-rank candidate recommendations by: minimum verification tier, academic vs. industry affiliation, current mentoring availability, and specific domain tags without re-invoking the full embedding pipeline.
Input:	Filter predicate parameters (e.g., `min_tier=2`, `is_available=true`).
Output:	Instantaneously updated recommendation list adhering to filter constraints.
Precondition:	Initial candidate recommendation set retrieved.
Postcondition:	Filtered results displayed in < 100ms.

4.5 Module 5: Multidisciplinary Team Formation and Gap Analysis

FR-TEAM-13: Multidisciplinary Project Decomposition

Priority:	[S] Should-Have
Description:	When a project is classified as interdisciplinary (e.g., AI in Poultry Farming), the system SHALL automatically decompose the requirements into discrete sub-domains (e.g., Sub-domain A: Computer Vision; Sub-domain B: Veterinary Pathology).
Input:	Interdisciplinary project description.
Output:	Decomposed sub-domain requirement tuples $(D_1, \mathcal{K}_1), (D_2, \mathcal{K}_2)$.
Precondition:	Project requirements contain multiple disparate high-level ontology roots.
Postcondition:	Dedicated recommendation tracks generated for each sub-domain.

FR-TEAM-14: Set-Cover Team Skill Gap Detection Engine

Priority:	[S] Should-Have
Description:	Given project requirements $\mathcal{K}_u$ and the set of accepted advisory members $\mathcal{M} = \{e_1, e_2, \ldots, e_m\}$, the system SHALL calculate the unresolved skill gap $\Delta\mathcal{K} = \mathcal{K}_u \setminus \bigcup_{e \in \mathcal{M}} \mathcal{K}_e$. If $\lvert\Delta\mathcal{K}\rvert > 0$, the platform SHALL automatically suggest complementary experts maximizing coverage of $\Delta\mathcal{K}$.
Input:	Selected team members and project requirement skills.
Output:	Missing skill matrix and suggested complementary recruits.
Precondition:	Student has assembled at least one team member.
Postcondition:	Gap visualizer highlights uncovered skill tags in red.

FR-TEAM-15: Side-by-Side Candidate Comparison Matrix

Priority:	[S] Should-Have
Description:	The system SHALL allow students to select up to 3 candidate experts and view an interactive, side-by-side comparison matrix evaluating: total publication count, verified credential tier, skill overlap percentage, response latency benchmark, and current student load.
Input:	Array of 2 to 3 candidate expert UUIDs.
Output:	Comparison table rendering comparative strengths and trade-offs.
Precondition:	Experts selected from recommendation explorer.
Postcondition:	Rendered comparison matrix with direct “Send Proposal” action buttons.

4.6 Module 6: Collaboration Lifecycle and Workspace Management

FR-COL-16: Collaboration Request Lifecycle State Machine (FSM)

Priority:	[M] Must-Have
Description:	The system SHALL enforce an auditable, rigorous state machine governing all collaboration invitations. Permitted states are: `SUBMITTED`, `UNDER_REVIEW`, `ACCEPTED`, `DECLINED`, `ACTIVE`, `COMPLETED`, and `CANCELLED`. Direct transitions outside the defined FSM graph SHALL be strictly rejected with HTTP 409 Conflict.
Input:	State transition triggers (Accept, Decline, Begin, Complete, Terminate).
Output:	Updated collaboration record and automated notification to all participants.
Precondition:	Request sent by authenticated student to verified expert.
Postcondition:	State transition persisted in database with audit timestamp.

FR-COL-17: Faculty Anti-Fatigue Quotas and Availability Blackouts

Priority:	[M] Must-Have
Description:	To prevent faculty burnout and proposal spam, the system SHALL enforce: • A monthly inbound collaboration proposal quota (default: max 5 pending requests per expert). • An automated blackout calendar toggle allowing experts to mark themselves unavailable. • A 14-day automated expiration timer on unanswered proposals (auto-transitions to `EXPIRED`).
Input:	Inbound collaboration request or expert blackout settings.
Output:	Rejection of excess requests with user-friendly quota notification.
Precondition:	Student attempts to dispatch proposal to expert.
Postcondition:	Request allowed only if expert quota is not saturated.

FR-COL-18: Secure Collaboration Workspace and Contextual Messaging

Priority:	[M] Must-Have
Description:	Once a collaboration reaches state `ACCEPTED`, the system SHALL automatically instantiate a private collaboration workspace containing: milestone tracking checklist, shared document repository, and contextual messaging.
Input:	Workspace messages, file attachments (PDF/ZIP, max 10MB), milestone updates.
Output:	Real-time synchronized message thread and milestone progress bar.
Precondition:	Collaboration state is `ACCEPTED` or `ACTIVE`.
Postcondition:	Messages encrypted and stored with full audit trail.

4.7 Module 7: Feedback, Governance, and Administration

FR-GOV-19: Structured Post-Collaboration Mutual Peer Review

Priority:	[**S**] Should-Have
Description:	Upon reaching state `COMPLETED`, both student and expert SHALL be prompted to submit structured qualitative and quantitative feedback. The review assesses: communication responsiveness, technical mentorship quality, goal achievement, and professional conduct.
Input:	Likert-scale ratings (1–5 stars) across 4 dimensions and free-text commentary.
Output:	Aggregated reputation score updated in expert and student profiles.
Precondition:	Collaboration transitioned to `COMPLETED`.
Postcondition:	Reviews published after both parties submit (double-blind review mechanism).

FR-GOV-20: Administrative Verification Audit and Moderation Dashboard

Priority:	[**M**] Must-Have
Description:	The system SHALL provide departmental administrators with an administrative portal to: inspect pending credential verification queues, approve or reject institutional documentation, review flagged users or inappropriate solicitations, and inspect real-time system metrics.
Input:	Administrative decisions (Approve, Reject, Suspend, Ban).
Output:	Updated user status and immutable audit log entry.
Precondition:	Authenticated with `ADMIN` scope.
Postcondition:	Administrative action logged with timestamp, admin ID, and reason.

5 Non-Functional Requirements (NFRs)

Non-functional requirements specify qualitative benchmarks and architectural constraints necessary to ensure industrial-grade software engineering standards.

5.1 Performance and Latency Requirements

Table 4: Performance Benchmarks and Acceptance Criteria

Req ID	Metric	Specification Criterion	Evaluation Method
NFR-PERF-01	Semantic Search Latency	95th percentile (P_{95}) search response time SHALL NOT exceed 250 milliseconds for a corpus of 10,000 expert vectors.	Automated Load Test (k6)
NFR-PERF-02	NLP Extraction Throughput	Zero-shot skill extraction for a 500-word proposal SHALL complete within 1.2 seconds on standard CPU.	Microbenchmark
NFR-PERF-03	System Concurrency	The backend SHALL sustain a minimum of 200 concurrent active HTTP requests without error rate exceeding 0.1%.	Stress Test
NFR-PERF-04	Page Load Time	First Contentful Paint (FCP) of the web client SHALL be $<$ 1.0s over a standard 4G broadband connection.	Google Lighthouse

5.2 Security and Confidentiality Requirements

- **NFR-SEC-01 (Transport Security):** All network data in transit SHALL be strictly enforced under TLS 1.3 encryption with HTTP Strict Transport Security (HSTS) headers enabled.
- **NFR-SEC-02 (Credential Hashing):** User passwords SHALL be salted and hashed using Argon2id with recommended OWASP memory cost parameters ($\geq 64\text{MB}, t = 3, p = 4$). Plaintext passwords SHALL never be logged or persisted.
- **NFR-SEC-03 (Token Security):** JWT access tokens SHALL be cryptographically signed using asymmetric Ed25519 or HMAC-SHA256 with an ephemeral key rotated every 30 days. Refresh tokens SHALL be stored exclusively in `HttpOnly`, `SameSite=Strict`, `Secure` cookies.
- **NFR-SEC-04 (Input Sanitization):** All user inputs SHALL be sanitized against SQL Injection (via asyncpg parameterized queries) and Cross-Site Scripting (XSS via DOMPurify / React automatic escaping).
- **NFR-SEC-05 (Rate Limiting):** Public authentication endpoints SHALL be throttled by Redis token-bucket rate limiters to a maximum of 5 failed attempts per minute per IP address.

5.3 Reliability and Availability Requirements

- **NFR-REL-01 (System Availability):** The core discovery and collaboration service SHALL maintain a minimum monthly uptime of 99.9% (excluding scheduled maintenance windows).

- **NFR-REL-02 (Data Durability):** PostgreSQL relational data SHALL be backed up daily with Write-Ahead Logging (WAL) archiving, achieving a Recovery Point Objective (RPO) $\leq$ 1 hour and Recovery Time Objective (RTO) $\leq$ 30 minutes.
- **NFR-REL-03 (Graceful Degradation):** In the event of temporary Qdrant vector store unavailability, the system SHALL automatically degrade to classical PostgreSQL full-text search (BM25 baseline) while alerting administrators.

5.4 Usability and Accessibility Requirements

- **NFR-USE-01 (Accessibility Standards):** The web application SHALL comply with WCAG 2.1 Level AA accessibility criteria, supporting keyboard-only navigation and full screen-reader compatibility.
- **NFR-USE-02 (System Usability Scale):** The student and expert interface SHALL achieve a System Usability Scale (SUS) score $\geq$ 80 during User Acceptance Testing (UAT).
- **NFR-USE-03 (Error Messaging):** All error responses presented to the user SHALL be actionable and written in clear, plain language, avoiding raw stack traces or internal database schemas.

6 Data Requirements and Conceptual Schema

6.1 Conceptual Data Model

The core domain model is structured around 8 primary entities managed under strict foreign key constraints and ACID transactional boundaries. Figure 3 outlines the high-level entity relationships.

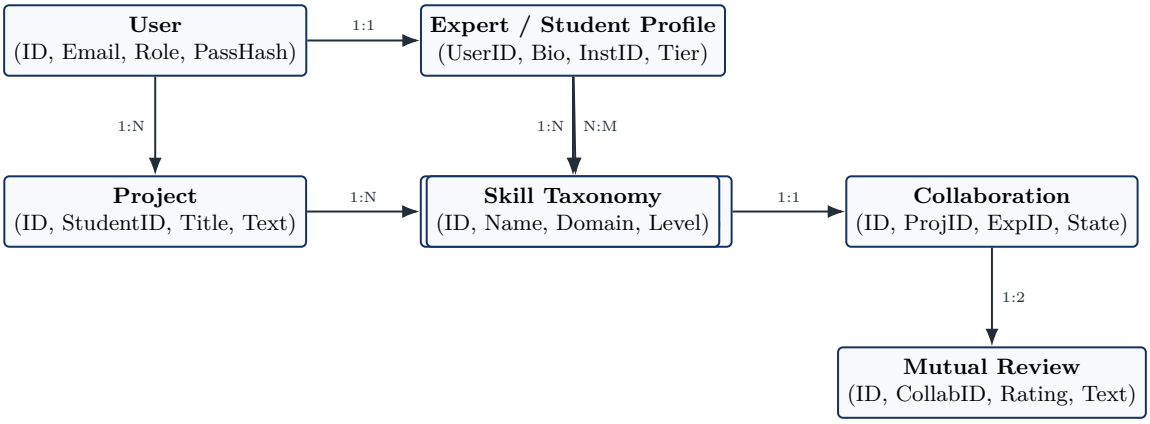


Figure 3: UniExpert Conceptual Entity-Relationship Diagram

6.2 Data Dictionary

Table 5 specifies the data dictionary for the core relational tables.

Table 5: Relational Database Data Dictionary

Entity / Table	Attribute Name	Data Type	Constraints & Description
`users`	`id`	UUID	Primary Key, generated via `uuid_generate_v4()`.
	`email`	VARCHAR(255)	Unique, valid email format, indexed.
	`password_hash`	VARCHAR(255)	Argon2id hashed password string.
	`role`	ENUM	`STUDENT`, `EXPERT`, `ADMIN`.
	`is_verified`	BOOLEAN	Default `FALSE`; set `TRUE` upon token activation.
`expert_profiles`	`id`	UUID	Primary Key; Foreign Key referencing `users(id)`.
	`institution`	VARCHAR(255)	Affiliated academic or corporate institution.
	`department`	VARCHAR(255)	Departmental unit or lab name.
	`verification_tier`	INTEGER	Values $1, 2, 3, 4$ representing badge tier.
	`availability_status`	ENUM	`AVAILABLE`, `LIMITED`, `BLACKOUT`.
	`max_active_load`	INTEGER	Default 3; maximum concurrent collaborations.
`projects`	`id`	UUID	Primary Key.
	`student_id`	UUID	Foreign Key referencing `users(id)`.
	`title`	VARCHAR(255)	Project working title.
	`raw_statement`	TEXT	Unstructured proposal text (50–2,000 words).
	`embedding_id`	VARCHAR(64)	Vector UUID mapped to Qdrant collection point.
`collaborations`	`id`	UUID	Primary Key.
	`project_id`	UUID	Foreign Key referencing `projects(id)`.
	`expert_id`	UUID	Foreign Key referencing `expert_profiles(id)`.
	`state`	ENUM	`SUBMITTED`, `ACCEPTED`, `ACTIVE`, `COMPLETED`, etc.
	`created_at`	TIMESTAMPTZ	Automatically stamped timestamp with timezone.

7 Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix (Table 6) correlates each functional requirement to its design module, test case ID, and verification method. Verification methods are classified per IEEE standard: **T** (Test), **D** (Demonstration), **A** (Analysis), and **I** (Inspection).

Table 6: Requirements Traceability Matrix (RTM)

Req ID	Requirement Title	Architectural Module	Test Case ID	Method
FR-AUTH-01	User Registration & Verification	Auth Service	TC-AUTH-01	T
FR-AUTH-02	Role-Based Access Control (RBAC)	Gateway Service	TC-AUTH-02	T
FR-PROF-03	Student Profile Management	Profile Service	TC-PROF-01	D
FR-PROF-04	Multi-Tier Credential Verification	Audit Service	TC-PROF-02	I, T
FR-PROF-05	Academic Skill Taxonomy	Taxonomy Service	TC-TAX-01	T
FR-NLP-06	Project Statement Ingestion	Project Service	TC-PROJ-01	D
FR-NLP-07	Automated Skill Entity Extraction	AI Inference Engine	TC-NLP-01	A, T
FR-NLP-08	Interactive Requirement Refinement	Web Client UI	TC-NLP-02	D
FR-REC-09	Dense Embedding Generation	AI Inference Engine	TC-VEC-01	T
FR-REC-10	Multi-Criteria Hybrid Scoring	Matchmaking Service	TC-REC-01	A, T
FR-REC-11	Explainable AI Justification	XAI Generator	TC-REC-02	D, T
FR-REC-12	Dynamic Faceted Filtering	Search Service	TC-REC-03	T
FR-TEAM-13	Multidisciplinary Decomposition	Team Service	TC-TEAM-01	A, T
FR-TEAM-14	Set-Cover Skill Gap Detection	Team Service	TC-TEAM-02	A, T
FR-TEAM-15	Side-by-Side Comparison Matrix	Web Client UI	TC-TEAM-03	D
FR-COL-16	Collaboration Lifecycle FSM	Collaboration Service	TC-COL-01	T
FR-COL-17	Anti-Fatigue Quotas & Blackouts	Governance Service	TC-COL-02	T
FR-COL-18	Secure Workspace & Messaging	Workspace Service	TC-COL-03	D, T
FR-GOV-19	Structured Peer Review	Reputation Service	TC-GOV-01	D
FR-GOV-20	Admin Verification Dashboard	Admin Service	TC-GOV-02	D, I

8 Verification and Validation Plan

8.1 Unit and Integration Testing Protocol

All software components shall be continuously verified within an automated GitHub Actions CI/CD pipeline executing:

(1) Unit Tests (Pytest): Minimum 85% line coverage across backend services, validating mathematical scoring functions, FSM transition integrity, and token parsing.
(2) API Integration Tests (HTTPX): End-to-end testing of REST endpoints ensuring compliance with JSON API schemas and status code contracts.
(3) Frontend Component Tests (Jest / React Testing Library): Verifying accessibility standards and component rendering across edge states.

8.2 Information Retrieval (IR) Benchmarking Protocol

The recommendation quality of UniExpert shall be quantitatively validated using a curated benchmark of 50 student research statements with ground-truth expert relevance judgments evaluated by 3 independent academic judges:

- **Precision at K ($P@K$):** Ratio of relevant experts among the top-K recommendations ($K = 3, 5$).
- **Normalized Discounted Cumulative Gain ($NDCG@K$):** Evaluates graded relevance taking position discount into account.
- **Comparative Baselines:** Performance of the Hybrid Engine (Eq. 1) shall be compared directly against BM25 lexical keyword matching and isolated dense vector retrieval.

8.3 User Acceptance Testing (UAT)

A formal UAT phase will be conducted in December 2026 involving a cohort of 20 university students and 8 faculty advisors. Test scenarios will cover: (1) project submission; (2) reviewing recommendations; (3) sending collaboration proposals; and (4) evaluating workspace utility. Success is defined as achieving a task completion rate $\geq 90\%$ and an average SUS score ≥ 80.

9 Committee Review and Sign-Off

By signing below, the supervisor and capstone examination committee confirm that this Software Requirements Specification (SRS) establishes the approved functional, architectural, and evaluation scope for the **UniExpert** Capstone Project for the Fall 2026 academic semester.

Mr. Md. Shohel Arman
Assistant Professor
Department of Software Engineering
Faculty of Science & Information Technology
Date: _____

Head of Department / Committee Chair
Department of Software Engineering
Faculty of Science & Information Technology
Date: _____